Manejo de Errores en APIS

- Manejo de Errores en APIS
  - Buenas prácticas
    - Códigos de estado HTTP
    - Estructura clara y consistente
    - Mensajes de error descriptivos
    - No incluir datos sensibles
    - Que no incluir en la respuesta
    - Documentar Errores comunes
    - Bitácora y Monitoreo

Buenas prácticas

Independientemente de la arquitectura en que este implementada nuestra API existen algunas buenas prácticas que deberemos de seguir para manejar los errores que se pudieran presentar.

- **Códigos de estado HTTP**
- **Estructura clara y consistente**
- **Mensajes de error descriptivos**
- **No incluir datos sensibles**
- **Bitácora y Monitoreo**

Códigos de estado HTTP

REST usa los códigos de estado del protocolo HTTP. Debemos asegurarse de responder con el código de error apropiado en la respuesta, de acuerdo al error que haya ocurrido.

```
HTTP/1.1 404 Not Found
Date: Fri, 11 Sep 2026 12:23:00 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 108
Connection: close

{
  "status": 404,
  "error": "Not Found",
  "message": "El recurso solicitado con ID '9876' no fue encontrado."
}
```

Estructura clara y consistente

Cuando se produzca un error la API deberá responder siempre con una estructura
uniforme de errores que sea consistente en toda la API.
Crear una estructura que se estándar para todos los posibles errores.
por ejemplo podemos contestar con un objeto de error que tenga las siguientes propiedades:

```
{
    "Error": "Código de error"
    "Mensaje": "Descripción del error"
    "Detalles": "Posibles soluciones"
}
```

Los errores devueltos por la API pueden seguir el formato RFC 7807 (Problem Details) en JSON:

```
{
  "type": "https://api.ejemplo.com/v1/errores/autenticacion-invalida",
  "title": "Token de autenticación no válido",
  "status": 401,
  "detail": "El token proporcionado ha expirado o no es válido. Genere uno nuevo",
  "instance": "/v1/usuarios/123",
  "code": "AUTH_INVALID_TOKEN"
}
```

Mensajes de error descriptivos

Los mensajes deben ser claros y dar una descripción clara del error
Con ese mensaje el cliente debe entender el error y como puede solucionarlo
Debe de incluir toda la información necesaria para que el cliente entienda
el error y como solucionarlo, por ejemplo:

```
{
  "type": "https://api.ejemplo.com/v1/errores/validacion",
  "title": "Error de validación en la petición",
  "status": 400,
  "code": "INVALID_INPUT",
  "message": "Los datos enviados no son válidos. Por favor revise el listado de errores.",
  "errors": [
    { "field": "email",
      "reason": "El correo electrónico debe ser una dirección válida (ej. usuario@dominio.com)."
    },
    { "field": "edad",
      "reason": "La edad debe ser un número entero mayor o igual a 18."
    }
  ],
  "timestamp": "2026-09-11T12:40:44Z",
  "traceId": "req-8f92a11b-3c4e"
}
```

No incluir datos sensibles

Tener cuidado de no incluir información sensible en el mensaje de error.
Tampoco deberemos incluir los mensajes de error técnicos, por ejemplo mensajes que regrese el intérprete, compilador o manejador de bases de datos.
Solo se debe incluir información del problema y como solucionarlo

```
{
  "error": "Internal Server Error",
  "detail": "Error: Connect ECONNREFUSED 127.0.0.1:5432 at /app/src/db.js:42\n    at Client._connectionCallback
(/app/node_modules/pg/lib/client.js:123)\n    at select * from users where usr_pass = 'secret123'"
}
```

Esto es exponencialmente Inseguro ya que estamos exponiendo:

- la ruta del código (/app/src/db.js),
- tecnología interna (pg, PostgreSQL), puertos, IPs)
- datos sensibles (contraseñas/consultas SQL).

Que no incluir en la respuesta

- Versiones de software
- Tecnologías o frameworks utilizados Nombres de servidores rutas internas, IPs
- Base de datos utilizada estructura nombres de tablas
- Mensajes de Error de Excepción o Run Time Error.

Respuesta Insegura

```
HTTP/1.1 500 Internal Server Error
Server: nginx/1.18.0 (Ubuntu)
X-Powered-By: Express

{
  "error": "Database error",
  "details": "SequelizeConnectionRefusedError: failed to connect to postgresql://admin:pass@db-cluster-
prod.internal:5432/main_db"
}
```

Respuesta segura:

```
HTTP/1.1 500 Internal Server Error
Server: Server

{
  "status": 500,
  "code": "INTERNAL_SERVER_ERROR",
  "message": "Ocurrió un error inesperado al procesar la solicitud.",
  "traceId": "req-9942a7b8"
}
```

Documentar Errores comunes

En la documentación de la API se puede incluir la estructura del mensaje de error que responde la API asi como una sección con los errores mas comunes y su solución, en una ruta que se llame /Troubleshooting /errores/

Código Http	Código API	Descripción	Solución Recomendada
400	INVALID_JSON	El cuerpo de la petición contiene un JSON mal formado	Revisa la sintaxis del JSON (comas sobrantes, corchetes o comillas)
401	TOKEN_EXPIRED	El token ha superado su tiempo de vida.	Renueva el token utilizando el endpoint `/v1/auth/refresh`
403	PERMISSIONS	El token es válido pero no tiene permisos para esta acción.	Verifica los scopes asignados a tus credenciales
404	RESOURCE_NOT_FOUND	El ID del recurso especificado en la URL no existe.	Confirma que el identificador existe o que la URL está bien escrita
422	VALIDATION_ERROR	Los datos son correctos pero violan reglas de negocio	Revisa el objeto errors en la respuesta para corregir los campos requeridos.
429	RATE_LIMIT	Se ha superado el número de peticiones permitidas por minuto.	Reintente mas tarde .
500	SERVER_ERROR	Ocurrió un error no esperado en nuestros servidores.	Reintente la petición. Si persiste, contacta al soporte

Bitácora y Monitoreo

Ademas de implementar una bitácora (log) que registre las llamadas a la API
Se recomienda implementar otra que registre los errores que ocurren.

```
// Ruta normal
app.get('/', (req, res) => {
  next(new Error('¡Ocurrió un error simulado!'));
});

// Middleware de manejo de errores
app.use((err, req, res, next) => {
  const logMensaje = `[${new Date().toISOString()}] ${req.method} ${req.url} - ${err.stack}\n\n`;

  // Guarda el error en el archivo error.log
  fs.appendFile(path.join(__dirname, 'error.log'), logMensaje, (fsErr) => {
    if (fsErr) console.error('Error escribiendo en el log:', fsErr);
  });

  // Respuesta al cliente
  res.status(500).json({ status: 500, error: 'Error interno del servidor' });
});

app.listen(PORT, () => console.log(`Servidor en http://localhost:${PORT}`));
```